

東吳大學 資訊管理學系

114 學年度畢業專題成果報告書

專題名稱：Dot to Dot (SCU Connect) - 東吳大學校園服務平台

Project Title: Dot to Dot (SCU Connect) - Campus Service Platform

指導教授：[指導教授姓名] 組員：[組員姓名 1] (學號) [組員姓名 2] (學號) [組員姓名 3] (學號) [組員姓名 4] (學號)

中華民國 114 年 12 月 4 日

目錄 (Table of Contents)

1. [專案概述 \(Project Overview\)](#)
 - [1.1 背景與動機](#)
 - [1.2 專案目標](#)
 - [1.3 三大核心服務](#)
2. [系統架構 \(System Architecture\)](#)
 - [2.1 架構圖說明](#)
 - [2.2 技術棧清單](#)
3. [技術規格 \(Technical Specifications\)](#)
 - [3.1 前端規格](#)
 - [3.2 後端規格](#)
4. [核心功能說明 \(Core Functionalities\)](#)
 - [4.1 註冊與登入](#)
 - [4.2 共乘服務](#)
 - [4.3 搬家/物流服務](#)
 - [4.4 跑腿服務](#)
 - [4.5 即時聊天](#)
 - [4.6 評分系統](#)
5. [資料庫設計 \(Database Design\)](#)
 - [5.1 實體關係圖](#)
 - [5.2 資料表結構詳解](#)
6. [API 端點規格 \(API Endpoint Specifications\)](#)
 - [6.1 身份驗證](#)
 - [6.2 共乘](#)
 - [6.3 物流](#)
 - [6.4 跑腿](#)
 - [6.5 管理員](#)
7. [使用者介面 \(User Interface\)](#)
 - [7.1 主要頁面](#)
 - [7.2 關鍵元件](#)
8. [安全性與驗證機制 \(Security & Verification\)](#)
 - [8.1 帳號安全](#)
 - [8.2 權限控制](#)
9. [系統管理功能 \(System Management\)](#)
10. [測試與驗證 \(Testing & Verification\)](#)
 - [10.1 功能測試清單](#)
 - [10.2 已驗證項目](#)
11. [未來展望 \(Future Prospects\)](#)
 - [11.1 短期規劃](#)
 - [11.2 中期規劃](#)
 - [11.3 長期規劃](#)
12. [結論 \(Conclusion\)](#)

1. 專案概述 (Project Overview)

1.1 背景與動機

東吳大學 (SCU) 擁有獨特的校園地理環境與學生生活型態。外雙溪與城中兩校區之間的交通往返、住宿生與租屋族的搬家需求，以及校園周邊餐飲的獲取便利性，一直是學生們日常生活中的痛點。現有的解決方案往往分散且缺乏針對校園場景的優化。

1.2 專案目標

本專案「Dot to Dot (SCU Connect)」旨在建立一個專屬於東吳大學的校園共享經濟服務平台。透過整合校園內的閒置資源（如機車座位、空閒時間、勞動力），以互助的形式解決交通、物流與生活瑣事，同時促進校園內的社交連結。

1.3 三大核心服務

- **共乘媒合 (Ride Sharing)****：解決兩校區通勤與返鄉交通問題，媒合機車/汽車駕駛與乘客。
- **校園物流 (Campus Logistics)****：提供宿舍搬家與大型物品運送媒合，建立校園內的即時物流網絡。
- **跑腿服務 (Errands & Delivery)****：提供校園餐飲代購與生活跑腿服務，解決學生日常生活瑣事。

2. 系統架構 (System Architecture)

本系統採用現代化的前後端分離架構，確保系統的擴充性與維護性。

2.1 架構圖說明

- **Client Side (前端)****：使用 React 框架建構單頁式應用 (SPA)，透過 RESTful API 與後端溝通，並使用 Socket.io Client 處理即時通訊。
- **Server Side (後端)****：基於 Node.js Runtime，使用 Express 框架處理 HTTP 請求與業務邏輯。
- **Database (資料庫)****：使用 SQLite 嵌入式關聯式資料庫，輕量且易於部署。
- **Real-time Service (即時服務)****：Socket.io Server 處理聊天室訊息廣播。
- **Email Service (郵件服務)****：整合 Gmail SMTP 發送驗證信件。

2.2 技術棧清單

類別	技術/工具	版本/說明
前端框架	React	v19.2.0
建置工具	Vite	v7.2.4
路由管理	React Router	v7.1.1
樣式設計	CSS3 / Vanilla CSS	原生 CSS 變數與 Flex/Grid 佈局
後端框架	Express	v5.1.0
執行環境	Node.js	v18+
資料庫	SQLite3	v5.1.7

即時通訊	Socket.io	v4.8.1
身份驗證	Bcrypt	密碼雜湊加密
郵件服務	Nodemailer	SMTP 郵件發送

3. 技術規格 (Technical Specifications)

3.1 前端規格

- SPA 架構：透過 React Router 實現無刷新頁面切換。
- **響應式設計 (RWD)**：支援桌機與行動裝置瀏覽。
- 元件化開發：封裝 Navbar, Card, Modal 等共用元件。

3.2 後端規格

- RESTful API：遵循標準 HTTP 方法 (GET, POST, PUT, DELETE)。
- Middleware：
 - cors：處理跨域資源共享。
 - body-parser：解析 JSON 請求主體。
 - isAdmin：自定義權限驗證中間件。
- 安全性：
 - 密碼儲存採用 bcrypt 加鹽雜湊。
 - API 參數驗證與錯誤處理。

4. 核心功能說明 (Core Functionalities)

4.1 註冊與登入 (Authentication)

- 學號驗證：限制僅能使用 @scu.edu.tw 結尾之信箱註冊，確保用戶皆為東吳學生。
- 驗證碼機制：系統自動發送 6 位數驗證碼至信箱，驗證有效性為 10 分鐘。
- 角色區分：支援一般用戶 (User) 與管理員 (Admin) 角色。

4.2 共乘服務 (Ride Sharing)

- 發布行程：駕駛可設定出發地、目的地、時間、座位數與價格。
- 行程搜尋：乘客可依起訖點篩選行程。
- 預訂機制：採用即時預訂確認機制，系統自動更新剩餘座位。

4.3 搬家/物流服務 (Logistics/Movers)

- 需求發布：用戶填寫物品類型、所需車型、起訖點與願付價格。
- 司機接單：具備運送能力的服務提供者可查看需求列表並接單。
- 狀態追蹤：訂單狀態流轉 (Open -> Accepted -> Completed)。

4.4 跑腿服務 (Errands)

- 任務發布：包括代買餐點、代領包裹等即時生活服務需求。
- 跑腿者接單：有意願提供服務的用戶可接單執行。

4.5 即時聊天 (Real-time Chat)

- 專屬聊天室：每個任務（共乘、搬家、跑腿）建立獨立聊天室。
- 即時訊息：透過 WebSocket 實現訊息即時送達，無須重新整理頁面。

4.6 評分系統 (Rating System)

- 雙向評分：服務完成後，雙方可互相評分（1-5 星）與留言。
- 信用累積：用戶個人頁面顯示平均評分與評價數。

5. 資料庫設計 (Database Design)

系統共包含 9 個主要資料表，採用關聯式設計。

5.1 實體關係圖 (ER Diagram 概述)

- Users 為核心實體，與所有服務表（Rides, Deliveries, Errands）透過 user_id（發起者）關聯。
- 服務表另有 driver_id / runner_id 關聯至 Users（執行者）。
- Chats 與 Ratings 為多型關聯，透過 type 與 id 對應不同服務。

5.2 資料表結構詳解

1. users (使用者)

欄位	類型	說明
id	INTEGER	PK, Auto Increment
email	TEXT	Unique, SCU Email
password	TEXT	Hashed Password
name	TEXT	顯示名稱
role	TEXT	'user' 或 'admin'
rating	REAL	平均評分
rating_count	INTEGER	評分次數

2. email_verifications (Email 驗證)

欄位	類型	說明
email	TEXT	PK
code	TEXT	6 位數驗證碼
expiresAt	DATETIME	過期時間

3. rides (共乘行程)

欄位	類型	說明
id	INTEGER	PK

user_id	INTEGER	FK -> users.id (駕駛)
origin	TEXT	出發地
destination	TEXT	目的地
departureTime	TEXT	出發時間
seats	INTEGER	座位數
price	INTEGER	價格

4. deliveries (搬家/物流)

欄位	類型	說明
id	INTEGER	PK
user_id	INTEGER	FK -> users.id (需求者)
driver_id	INTEGER	FK -> users.id (司機)
item_type	TEXT	物品類型
required_vehicle	TEXT	需求車輛
status	TEXT	'open', 'accepted', 'completed'

5. errands (跑腿任務)

欄位	類型	說明
id	INTEGER	PK
user_id	INTEGER	FK -> users.id (需求者)
runner_id	INTEGER	FK -> users.id (跑腿者)
item	TEXT	任務內容
shop_location	TEXT	購買/執行地點
meet_location	TEXT	交貨地點
price	INTEGER	跑腿費

6. chats (聊天訊息)

欄位	類型	說明
id	INTEGER	PK
type	TEXT	'errand', 'ride', 'mover'

related_id	INTEGER	對應的任務 ID
sender_id	INTEGER	FK -> users.id
content	TEXT	訊息內容

7. notifications (通知 - 預留)

欄位	類型	說明
id	INTEGER	PK
user_id	INTEGER	接收者
content	TEXT	通知內容
is_read	BOOLEAN	是否已讀

8. ratings (評價)

欄位	類型	說明
id	INTEGER	PK
from_user_id	INTEGER	評分者
to_user_id	INTEGER	被評分者
score	REAL	分數 (1-5)
comment	TEXT	評語

6. API 端點規格 (API Endpoint Specifications)

6.1 身份驗證 (Auth)

- POST /api/send-code : 發送註冊驗證碼。
- POST /api/register : 註冊新帳號 (需驗證碼)。
- POST /api/login : 使用者登入。

6.2 共乘 (Rides)

- GET /api/rides : 取得所有行程 (支援 from , to 篩選)。
- POST /api/rides : 發布新行程。
- PUT /api/rides/:id : 修改行程。
- DELETE /api/rides/:id : 刪除行程。

6.3 物流 (Deliveries)

- GET /api/deliveries : 取得所有物流需求。
- POST /api/deliveries : 發布物流需求。
- PUT /api/deliveries/:id : 修改需求。

- `DELETE /api/deliveries/:id` : 刪除需求。

6.4 跑腿 (Errands)

- `GET /api/errands` : 取得所有跑腿任務。
- `POST /api/errands` : 發布跑腿任務。
- `PUT /api/errands/:id/accept` : 接單 (Runner)。

6.5 管理員 (Admin)

- `GET /api/admin/stats` : 取得系統統計數據。
- `GET /api/admin/users` : 取得所有用戶列表。
- `DELETE /api/admin/users/:id` : 刪除用戶。
- `GET /api/admin/tasks/:type` : 取得特定類型的所有任務。
- `DELETE /api/admin/tasks/:type/:id` : 強制刪除任務。

7. 使用者介面 (User Interface)

7.1 主要頁面

1. **Home (首頁)** : 服務入口導航，展示三大服務區塊。
2. **Login / Register (登入/註冊)** : 包含學號輸入、驗證碼發送與密碼設定。
3. **Profile (個人頁面)** : 顯示個人資料、評分、發布的歷史紀錄。
4. **Transport (共乘大廳)** : 顯示行程列表卡片，提供搜尋與篩選功能。
5. **PostRide (發布行程)** : 表單介面，設定行程細節。
6. **Logistics (物流大廳)** : 顯示搬家/運送需求列表。
7. **PostDelivery (發布物流)** : 表單介面，上傳需求細節。
8. **Errands (跑腿大廳)** : 顯示跑腿任務列表。
9. **PostErrand (發布跑腿)** : 表單介面。
10. **ChatRoom (聊天室)** : 即時通訊介面，顯示歷史訊息。
11. **AdminDashboard (管理後台)** : 管理員專用，含數據總覽與管理列表。

7.2 關鍵元件

- **Navbar** : 響應式導覽列，依登入狀態顯示不同選項。
- **ServiceCard** : 統一風格的卡片元件，用於展示各類任務摘要。
- **StatusBadge** : 顯示任務狀態 (Open/Accepted) 的標籤。

8. 安全性與驗證機制 (Security & Verification)

8.1 帳號安全

- **密碼加密** : 使用 `bcrypt` 進行 Salted Hash 處理，資料庫不儲存明碼。
- **校園驗證** : 強制綁定 `@scu.edu.tw` 信箱，確保社群單純性與安全性。

8.2 權限控制

- **前端路由保護** : 未登入用戶無法訪問發布頁面與個人頁面。
- **後端 API 保護** :
 - 管理員 API 設有 `isAdmin` 中間件，驗證請求者角色。

- 修改/刪除功能驗證操作者是否為資源擁有者。

9. 系統管理功能 (System Management)

管理後台 (/admin) 提供三大模組：

1. 儀表板 (Dashboard)

- 即時顯示總用戶數、共乘行程數、物流單數、跑腿單數。
- 視覺化統計卡片。

2. 用戶管理 (User Management)

- 列表顯示所有註冊用戶資訊 (ID, Email, Name, Role)。
- 提供編輯功能：修改用戶姓名、權限 (升級 Admin)。
- 提供刪除功能：移除違規用戶。

3. 任務管理 (Task Management)

- 分頁籤檢視各類別 (Errands, Rides, Deliveries) 的所有任務。
- 顯示任務詳情與發起人。
- 管理員可強制刪除不當或過期的任務內容。

10. 測試與驗證 (Testing & Verification)

10.1 功能測試清單

- ☒ 註冊流程：驗證碼發送正常，資料寫入正確。
- ☒ 登入/登出：狀態保存與清除正常。
- ☒ 共乘功能：發布、搜尋、刪除流程無誤。
- ☒ 物流功能：發布需求、列表顯示正常。
- ☒ 跑腿功能：發布、接單狀態更新正常。
- ☒ 即時聊天：Socket 連線穩定，訊息即時同步。
- ☒ 管理後台：數據統計準確，刪除功能有效。

10.2 已驗證項目

- 確認 Email 服務可成功寄信至外部信箱。
- 確認 SQLite 資料庫在並發讀寫下的穩定性。
- 確認 React Router 在頁面跳轉時無破圖或狀態遺失。

11. 未來展望 (Future Prospects)

11.1 短期規劃 (1-3 個月)

- RWD 優化：針對手機版介面進行更細緻的調整，提升行動體驗。
- 圖片上傳：實作 AWS S3 或類似服務，讓搬家/跑腿可上傳物品照片。

11.2 中期規劃 (3-6 個月)

- 金流串接：整合 LinePay 或街口支付，提供線上付款與履約保證。

- 身份認證升級：整合學生證 OCR 或學校 SSO 登入。

11.3 長期規劃 (6-12 個月)

- 行動 App 開發：使用 React Native 移植為原生 App。
- AI 智慧媒合：分析用戶習慣，自動推薦合適的共乘夥伴或任務。

12. 結論 (Conclusion)

「Dot to Dot」專案成功建構了一個專屬於東吳大學的整合式校園服務平台。透過整合共乘、物流與跑腿三大需求，我們不僅解決了學生的生活痛點，更建立了一個互助的校園社群。

技術亮點

1. 完整的前後端整合：從資料庫設計到前端互動的完整實作。
2. 即時互動體驗：Socket.io 帶來的流暢溝通體驗。
3. 實用的管理系統：具備完整的後台管理機能，確保平台營運的可控性。

本專案展示了將現代 Web 技術應用於解決實際校園問題的潛力，並為未來的擴充與商業化奠定了堅實的基礎。

附錄：快速啟動指南

1. 安裝依賴：`npm install`
2. 啟動後端：`node server/index.js` (Port 3000)
3. 啟動前端：`npm run dev` (Port 5173)
4. 管理員帳號：`admin@scu.edu.tw` / `password123`